A unified second-order immersed-boundary and ghost-fluid method on a single Cartesian mesh for the Vlasov–Poisson system with complex boundaries

Hunt Feng[a,1], Haibo Dong[a,2], Chen Cui[a,3,*]

[a]Department of Mechanical and Aerospace Engineering, University of Virginia, 122 Engineer's Way, Charlottesville, 22904, VA, USA

Abstract

Plasma-material interactions are critical to applications from electric propulsion and spacecraft charging to fusion divertor erosion and semiconductor fabrication. Predictive modeling requires high-fidelity, fully kinetic simulations that resolve the velocity distribution function (VDF) near complex boundaries. Particle-in-Cell (PIC) methods are widely used but suffer from noise and poor sampling of high-energy tails, where key plasma–surface processes occur. Grid-based Vlasov method avoids these issues by directly solving the VDF, but faces challenges imposing sharp, curved geometries on Cartesian meshes without costly local refinement.

We present a unified, second-order immersed-boundary framework for the 2D2V Vlasov-Poisson system on a single Cartesian grid. A ghost-fluid method enforces dielectric jump conditions between plasma and solid in the Poisson solver, while a sharp-interface ghost-cell scheme applies complex geometry conditions for the Vlasov equation without staircasing errors or dual meshes. Coupled with a Positive Flux Conserving semi-Lagrangian scheme, our method ensures charge conservation, VDF positivity, and relaxed CFL limits.

This approach enables accurate, noise-free kinetic simulations in complex geometries, resolving high-energy tails and near-wall structures. Results on classical plasma-dielectric cases demonstrate robust handling of curved boundaries, opening a practical path for Vlasov modeling in real-world applications.

Keywords:

Semi-Lagrangian scheme, Immersed boundary method, Vlasov-Poisson system, Ghost

fluid method, Plasma-material interaction

*Corresponding author
Email address: cuichen@virginia.edu (Chen Cui)
[1]Visiting Researcher, University of Virginia
[2]Professor, University of Virginia
[3]Assistant Professor, University of Virginia

1. Introduction

Grid-based kinetic plasma simulations based on the Vlasov equation evolve the distribution function directly in phase space rather than sampling it with numerical particles. In contrast to particle-in-cell (PIC) methods, the continuum approach is free of sampling noise and therefore gives direct access to fine distribution-function structure, weak phase-space features, and wall fluxes. For that reason, continuum kinetic methods are especially attractive in bounded plasma problems where sheath structure, non-Maxwellian features, and plasma-material interaction must be resolved with high fidelity.

One of the main obstacles for applying the Vlasov method in the real engineering is geometric complexity. In a Vlasov solver, the physical-space mesh is coupled to a discretization in velocity space, and every additional cost in configuration space is effectively replicated across the velocity grid. As a result, design choices that are common in CFD, such as body-fitted meshes, heavy local refinement near curved walls, or distinct meshes for transport and field equations, can become much more expensive and much harder to manage in the grid-based kinetic computations. Recent high-order GPU work on Vlasov-Poisson solvers again highlights that the very large number of phase-space unknowns is a central practical limitation of the grid-based kinetic methods.

One possible approach for resolving this issue is to use body-fitted or unstructured meshes directly in phase space. This direction was explored early, for example, by Besse and Sonnendrücker, who developed semi-Lagrangian Vlasov schemes on unstructured phase-space meshes. However, the literature also makes clear why this route is difficult to scale in arbitrary geometry. Filbet and Yang explicitly argued that, because of the high-dimensional nature of kinetic equations, unstructured-mesh treatments are often unattractive and that Cartesian-grid techniques are preferable when possible. For Vlasov solvers, this argument is particularly compelling because many high-order Eulerian and semi-Lagrangian algorithms derive much of their efficiency from structured-grid operations.

On the other hand, a uniform mesh with arbitrary boundary condition can be a promising approach. More recently, Liu et al. developed an immersed finite element and conservative semi-Lagrangian framework for plasma-material interaction on structured interface-independent meshes, combining a first order half-way ghost-cell treatment for

Vlasov transport with an immersed field solve. These works clearly demonstrate progress, but they also show that the design space remains open, particularly for time-dependent, same-mesh, sharp-interface Vlasov–Poisson formulations with explicit near-boundary accuracy analysis.

A closely related body of work exists in deterministic rarefied-gas simulation, where discrete-velocity and BGK/Boltzmann solvers face a very similar tension between velocity-space cost and geometric fidelity. There, Cartesian embedded-boundary ideas have proved especially attractive. Mieussens' survey emphasized immersed Cartesian techniques as a promising direction for deterministic solvers. Filbet and Yang developed an inverse Lax-Wendroff boundary treatment for Boltzmann-type models in arbitrary geometry on Cartesian meshes. Dechriste and Mieussens later showed that conservative Cartesian cut-cell methods can combine geometric flexibility, exact conservation, and a relatively simple implementation for rarefied flows around moving obstacles. This rarefied-gas literature strongly suggests that a Cartesian sharp-interface treatment is a natural strategy for grid-based Vlasov solvers as well.

The contribution of the present work is therefore not simply to introduce complex geometry into Vlasov simulation, since that topic has already seen meaningful progress, but to provide a unified same-mesh formulation in which both the Vlasov and Poisson equations are treated with second-order sharp-interface accuracy near the boundary. This is particularly useful when the interior Vlasov solver is of higher order: a second-order near-boundary closure is accurate enough to remain consistent with the interior discretization on practical meshes, while avoiding the very dense local refinement that would otherwise be required around curved objects.

The remainder of the paper is organized as follows. Section 2 introduces the mathematical model and boundary conditions. Section 3 presents the unified same-mesh numerical method. Section 4 discusses the consistency and accuracy analysis near the immersed boundary. Section 5 contains numerical verification and validation. Section 6 presents representative application cases. Section 7 concludes the paper.

2. Governing Equations and Problem Setting

2.1. Vlasov-Poisson System

This part introduces the Vlasov-Poisson system

2.2. Nondimensionalization and geometry representation

2.3. Boundary conditions

Mathematically define the boundary condition for Poisson and Vlasov

3. Unified IBM and GFM Numerical Method

3.1. Governing equations

Assuming a collisionless plasma, the evolution of the distribution function for particle species s is governed by the Vlasov equation:

$$\partial_t f_s + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + \left(\frac{q_s}{m_s} \mathbf{E} \right) \cdot \nabla_{\mathbf{v}} f_s = 0. \tag{1}$$

The Vlasolver solves the Vlasov–Poisson system by coupling a kinetic solver for Eq. (1) with a field solver for the Poisson equation.

3.2. Unified coupling framework

We introduce a cell-centered Cartesian mesh (Fig. 1), which is shared by both the Vlasov and Poisson solvers. As illustrated in Fig. 1, both the distribution function and the electrostatic potential are defined at cell centers, enabling a consistent and efficient data exchange between the two solvers.

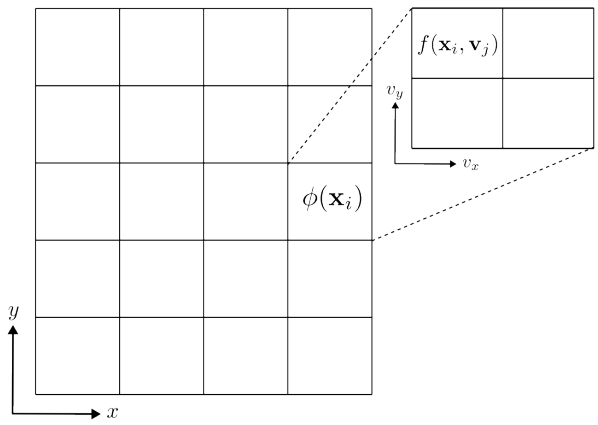

Figure 1: Schematic of the unified cell-centered mesh used for both the Vlasov and Poisson solvers.

The Vlasov equation is discretized using a semi-Lagrangian scheme based on the positive flux-conservative (PFC) formulation. The primary unknown is the cell-averaged distribution function:

$$f_{ij}^n = \frac{1}{\Delta x \Delta v} \int_{x_{i-1/2}}^{x_{i+1/2}} \int_{v_{j-1/2}}^{v_{j+1/2}} f(\mathbf{x}, \mathbf{v}, t^n) \, \mathrm{d}x \, \mathrm{d}v. \tag{2}$$

The scheme advances these cell averages directly, ensuring conservation and positivity without explicitly evaluating the integrals during time evolution.

The electrostatic potential is computed by solving the Poisson equation using a finite-difference method on the same mesh. The charge density is obtained from velocity moments of the distribution function:

$$\rho(\mathbf{x}) = \sum_s q_s \int f_s(\mathbf{x}, \mathbf{v})\, \mathrm{d}\mathbf{v}. \tag{3}$$

The overall coupling procedure is illustrated in Fig. 2. At each time step, the Vlasov solver updates the distribution function using the electric field, while the Poisson solver computes the electric field from the charge density. This establishes a self-consistent evolution of particles and fields.

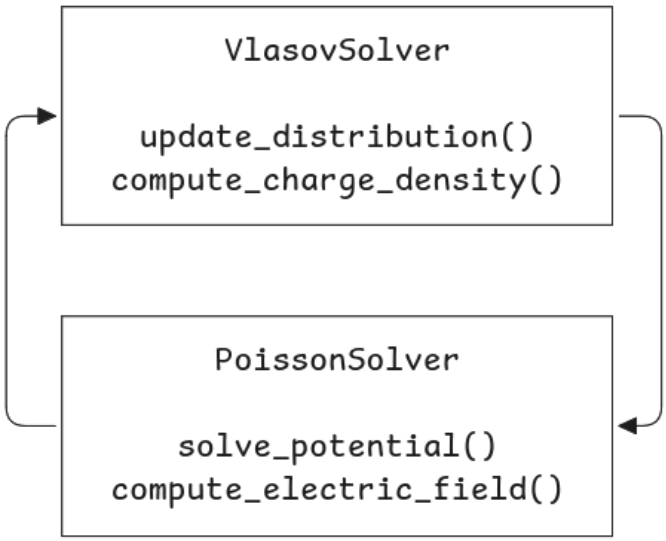

Figure 2: Flowchart of the coupling between the Vlasov and Poisson solvers in the Vlasolver framework.

3.3. Level set representation of immersed boundaries

The computational domain Ω is partitioned into fluid and solid regions by an immersed boundary which is implicitly represented by a level set function $\eta(\mathbf{x})$. With the level set function, each region can be defined as follows:

- Fluid region: $\Omega^+ = \{\mathbf{x} \in \Omega | \eta(\mathbf{x}) > 0\}$.
- Solid region: $\Omega^- = \{\mathbf{x} \in \Omega | \eta(\mathbf{x}) < 0\}$.
- Immersed boundary: $\Gamma = \{\mathbf{x} \in \Omega | \eta(\mathbf{x}) = 0\}$.

Moreover, the level set function allows us to compute the outward unit normal vector $\mathbf{n}$ at the interface Γ by

$$\mathbf{n} = \frac{\nabla \eta}{|\nabla \eta|}, \tag{4}$$

which is essential for enforcing boundary conditions in both the Vlasov and Poisson solvers.

3.4. Vlasov solver

A well known second order time integration technique called Strang splitting is applied to the Vlasov equation. This technique splits the Vlasov equation into two advection equations, one in the position space and the other in the velocity space.

$$\partial_t f + \mathbf{v}_s \cdot \nabla_{\mathbf{x}} f_s = 0 \tag{5}$$

$$\partial_t f + (q_s \mathbf{E}/m_s) \cdot \nabla_{\mathbf{v}} f_s = 0 \tag{6}$$

Then update the two advection equations in the following orders:

$$\begin{aligned} f_s^*(\mathbf{x}, \mathbf{v}) &= f_s^n(\mathbf{x} - \mathbf{v}_s \Delta t/2, \mathbf{v}), \\ f_s^{**}(\mathbf{x}, \mathbf{v}) &= f_s^*(\mathbf{x}, , \mathbf{v} - (q_s \mathbf{E}/m_s)\Delta t/2), \\ f_s^{n+1}(\mathbf{x}, \mathbf{v}) &= f_s^{**}(\mathbf{x} - \mathbf{v}_s \Delta t/2, \mathbf{v}). \end{aligned} \tag{7}$$

In each step, the advection update is performed using a 3rd-order PFC scheme.

3.5. PFC scheme for advection update

Notice that the spatial and velocity advection equations both take the form

$$\partial_t f + \partial_x (v f) = 0, \tag{8}$$

where v denotes as velocity $\mathbf{v}$ in Eq. (5) or acceleration $q\mathbf{E}/m$ in Eq. (6). A PFC scheme is implemented to solve such advection equation.

On a uniform cell-centered mesh defined on computational domain $[x_{\min}, x_{\max}]$. The cell centers are defined as $x_i = x_{\min} + (i + 1/2)\Delta x$, where Δx is the cell size. We use $f_i^n = \int_{x_{i-1/2}}^{x_{i+1/2}} f(x, t^n)\mathrm{d}x/\Delta x$ to denote the average distribution function at the i-th cell center and time step n.

The PFC scheme updates the distribution function at the next time step, f_i^{n+1}, by tracing the characteristic line back to the previous time step, t^n,

$$\int_{x_{i-1/2}}^{x_{i+1/2}} f(x, t^{n+1})\mathrm{d}x = \int_{x_{i-1/2}-v\Delta t}^{x_{i+1/2}-v\Delta t} f(x, t^n)\mathrm{d}x. \tag{9}$$

Meaning that the cell-averaged distribution function at the next time step, f_i^{n+1}, can be evaluated by

$$f_i^{n+1} = f_i^n + F_{i-1/2} - F_{i+1/2}. \tag{10}$$

where $F_{i-1/2} = \int_{x_{i-1/2}-v\Delta t}^{x_{i-1/2}} f(x, t^n)\mathrm{d}x/\Delta x$ and $F_{i+1/2} = \int_{x_{i+1/2}-v\Delta t}^{x_{i+1/2}} f(x, t^n)\mathrm{d}x/\Delta x$ are the fluxes through the cell left and right interfaces.

The flux can be approximated by a 3rd-order scheme,

$$F_{i+1/2} = \begin{cases} \varphi\left(f_i^n + \frac{1}{6}(2 - |\varphi|)(1 - |\varphi|)(f_{i+1}^n - f_i^n) + \frac{1}{6}(1 - |\varphi|)(1 + |\varphi|)(f_i^n - f_{i-1}^n)\right), & 1 > \varphi > 0, \\ \varphi\left(f_i^n + \frac{1}{6}(2 - |\varphi|)(1 - |\varphi|)(f_i^n - f_{i-1}^n) + \frac{1}{6}(1 - |\varphi|)(1 + |\varphi|)(f_{i+1}^n - f_i^n)\right), & -1 < \varphi < 0, \end{cases} \tag{11}$$

where $\varphi = v\Delta t/\Delta x$. If $\varphi > 1$, we can apply a whole grid shift to relax the CFL number.

3.6. Immersed boundary treatment for the Vlasov solver

The computational domain is partitioned into fluid, solid, and ghost cells. As shown in Fig. 3, ghost cells are solid cells adjacent to the interface that are used to enforce boundary conditions.

The interface is implicitly represented, and its intersection with grid lines defines interface points $\mathbf{x}_I$. At each interface point, the outward normal vector $\mathbf{n}$ is computed to distinguish inflow and outflow directions in velocity space.

The boundary condition is imposed directly on the distribution function at interface points. The treatment depends on the sign of $\mathbf{v} \cdot \mathbf{n}$:

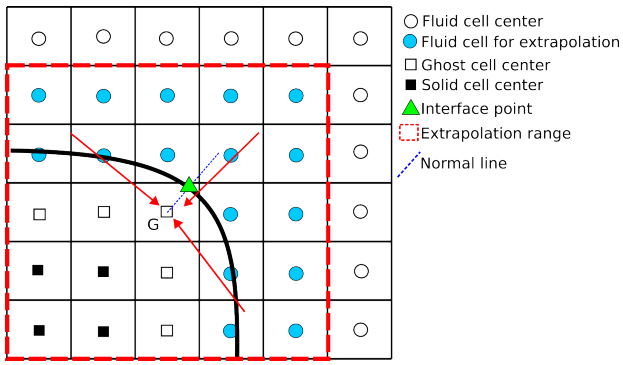

Figure 3: Illustration of cell classification and interface representation in the immersed boundary method.

- Outflow ($\mathbf{v} \cdot \mathbf{n} < 0$): The distribution function is extrapolated from interior fluid values.
- Inflow ($\mathbf{v} \cdot \mathbf{n} > 0$): A fully absorbing boundary condition is imposed, i.e., $f(\mathbf{x}_I, \mathbf{v}) = 0$.

This directional treatment ensures consistency with the kinetic transport mechanism and avoids unphysical reflections at the boundary.

Once the interface values are determined, ghost-cell values are reconstructed to complete the numerical stencil. As illustrated in Fig. 3, the ghost value is obtained by extrapolation using the nearby fluid cells.

Given a range for extrapolation (Fig. 3), the coefficients, c_0, c_1 and c_2 are determined by the distribution function at near-by fluid cells, which are selected based on the shape of the immersed object. We obtain the coefficients c_0, c_1 and c_2 by solving the least-square problem,

$$\min_{c_0,c_1,c_2} \sum_{k=1}^{N_F} (c_0 + c_1 x_k + c_2 y_k - f_k)^2 \tag{12}$$

where N_F is the number of near-by fluid cells in the extrapolation range, and f_k is the distribution function at the k-th near-by fluid cell.

For inflow directions, since we need to enforce zero-inflow condition on the boundary, the above least-square problem is solved with an additional constraint, $c_0 + c_1 x_I + c_2 y_I = 0$, where $(x_I, y_I) = \mathbf{x}_G - \eta \mathbf{n}$ is the coordinate of the interface point. The final extrapolation formula for ghost cells is given by

$$f(\mathbf{x}_G, \mathbf{v}, t) = c_0 + c_1 x_G + c_2 y_G \tag{13}$$

3.7. Boundary-aware positivity-preserving limiter for the Vlasov solver

To preserve positivity of the distribution function at fluid cells, a modified flux based on [1] is used.

$$\hat{F}_{i+1/2} = L_{i+1/2}\left(F_{i+1/2} - f_{i+1/2}\right) + f_{i+1/2}. \tag{14}$$

The first-order flux $f_{i+1/2}$ is defined to be boundary-aware,

$$f_{i+1/2} = \begin{cases} \varphi f_i & \text{if } f_i \geq 0, \\ 0 & \text{otherwise} \end{cases}. \tag{15}$$

In our simulation, the immersed boundary is fully absorbing, which means that the distribution function at the boundary is zero. Therefore, the first-order flux $f_{i+1/2}$ is designed to be consistent with the boundary condition by using the value of f_i when it is non-negative and setting it to zero otherwise.

The parameter $L_{i+1/2}$ is designed to ensure the positivity of the distribution function at the next time step, f_i^{n+1}, in Eq. (10). The idea is to limit the high-order flux $F_{i+1/2}$ and partially fallback to the first-order boundary-aware flux $f_{i+1/2}$ when the high-order flux would lead to negativity.

Finally, the new update scheme for the distribution function is

$$f_i^{n+1} = f_i^n + \hat{F}_{i-1/2} - \hat{F}_{i+1/2}. \tag{16}$$

To better describe the modified flux, let $d_{i\pm1/2} = F_{i\pm1/2} - f_{i\pm1/2}$, $\delta_i^n = 0 - (f_i^n + f_{i-1/2} - f_{i+1/2})$ and $p_i = d_{i-1/2} - d_{i+1/2} - \delta_i^n$. The upper bounds $\Lambda_i^r, \Lambda_{i+1}^l$ are defined as follows,

1. If $d_{i-1/2} \geq 0$ and $d_{i+1/2} \leq 0$,
$$(\Lambda_i^l, \Lambda_{i+1}^r) = (1, 1).$$
2. If $d_{i-1/2} \geq 0$ and $d_{i+1/2} > 0$,
$$(\Lambda_i^l, \Lambda_i^r) = \left(1, \min\left(1, \frac{\delta_i^n}{-d_{i+1/2}}\right)\right).$$
3. If $d_{i-1/2} < 0$ and $d_{i+1/2} \leq 0$,
$$(\Lambda_i^l, \Lambda_i^r) = \left(\min\left(1, \frac{\delta_i^n}{d_{i-1/2}}\right), 1\right).$$

4. If $d_{i-1/2} < 0$ and $d_{i+1/2} > 0$,

   - If $p_i < 0$,
$$(\Lambda_i^l, \Lambda_i^r) = \left(\frac{\delta_i^n}{d_{i-1/2} - d_{i+1/2}}, \frac{\delta_i^n}{(d_{i-1/2} - d_{i+1/2})} \right).$$
   - Else,
$$(\Lambda_i^l, \Lambda_i^r) = (1, 1).$$

Now that we have upper bounds Λ_i^l, Λ_i^r, the parameter $L_{i+1/2}$ in Eq. (14) is defined as

$$L_{i+1/2} = \begin{cases} \min(\Lambda_i^r, \Lambda_{i+1}^l) & \text{if } \eta_i > 0 \text{ and } \eta_{i+1} > 0 \\ \Lambda_i^r & \text{if } \eta_i > 0 \text{ and } \eta_{i+1} <= 0 \\ \Lambda_{i+1}^l & \text{if } \eta_i <= 0 \text{ and } \eta_{i+1} > 0 \end{cases} \tag{17}$$

3.8. Second-order ghost-fluid treatment for the Poisson equation

We consider the elliptic interface problem on a 2D domain $\Omega \subset \mathbb{R}^2$:

$$\nabla \cdot (\epsilon \nabla \phi) = -\rho \quad \text{in } \Omega \setminus \Gamma, \tag{18}$$

$$[\phi] = a \quad \text{on } \Gamma, \tag{19}$$

$$[\epsilon \partial_n \phi] = b \quad \text{on } \Gamma, \tag{20}$$

where the notation $[q] = q^+ - q^-$ denotes the jump.

Throughout the discussion, we assume the permittivity, ϵ, is piecewise constant and a, b are given functions defined on the interface. The normal vector components n_x and n_y are approximated by fourth-order central differences of the level set function η.

3.8.1. Finite Difference Discretization

Away from the interface, the equation is discretized using standard second-order central differences:

$$\begin{aligned} &\frac{1}{\Delta x}\left[\epsilon_{i+\frac{1}{2},j}\frac{\phi_{i+1,j} - \phi_{i,j}}{\Delta x} - \epsilon_{i-\frac{1}{2},j}\frac{\phi_{i,j} - \phi_{i-1,j}}{\Delta x}\right] \\ &+\frac{1}{\Delta y}\left[\epsilon_{i,j+\frac{1}{2}}\frac{\phi_{i,j+1} - \phi_{i,j}}{\Delta y} - \epsilon_{i,j-\frac{1}{2}}\frac{\phi_{i,j} - \phi_{i,j-1}}{\Delta y}\right] = -\rho_{i,j}. \end{aligned} \tag{21}$$

3.8.2. Interface Treatment via Boundary Condition Capturing

For each grid point (i, j) near the interface, define four directional points:

$$x_R = (x_i + \theta_R \Delta x, y_j), \quad x_L = (x_i - \theta_L \Delta x, y_j), \tag{22}$$

$$x_T = (x_i, y_j + \theta_T \Delta y), \quad x_B = (x_i, y_j - \theta_B \Delta y), \tag{23}$$

where $\theta \in (0, 1]$ gives the fractional distance to the interface, computed from the level set function using quadratic interpolation. For example, if $(x_i, y_j) \in \Omega^-$, then

$$\theta_R = \begin{cases} 1 & , \eta_{i+1,j}\eta_{i,j} > 0 \\ \frac{-D_x^0\eta + \sqrt{(D_x^0\eta)^2 - 4(D_{xx}^0\eta)\eta_{i,j}}}{2D_{xx}^0\eta} & , \eta_{i+1,j}\eta_{i,j} < 0 \end{cases}, \tag{24}$$

where $D_{xx}^0\eta = (\eta_{i-1,j} - 2\eta_{i,j} + \eta_{i+1,j})/2$ and $D_x^0\eta = (\eta_{i+1,j} - \eta_{i-1,j})/2$. For $\mathbf{x}_L, \mathbf{x}_T$ and $\mathbf{x}_B$, and the case where $(x_i, y_j) \in \Omega^+$ are computed similarly.

3.8.3. Second-Order Interface Reconstruction

At an interface point (e.g. $\mathbf{x}_R$), denote the one-sided values by ϕ_R^- and ϕ_R^+. These are determined by enforcing:

$$[\phi](\mathbf{x}_I) = a_I, \tag{25}$$

$$[\epsilon\phi_n](\mathbf{x}_I) = b_I, \tag{26}$$

where a_I is approximated by cubic interpolation of $b_{i-1}, b_i, b_{i+1}, b_{i+2}$ and so is b_I.

3.8.4. Modified Discrete Operator

The final discrete equation at (i, j) becomes:

$$\begin{aligned} &\left[\epsilon_{i+\theta_R/2,j}\frac{\phi_R - \phi_{i,j}}{\theta_R \Delta x} - \epsilon_{i-\theta_L/2,j}\frac{\phi_{i,j} - \phi_L}{\theta_L \Delta x}\right] \Big/ \left(\frac{\theta_R + \theta_L}{2}\Delta x\right) \\ &+ \left[\epsilon_{i,j+\theta_T/2}\frac{\phi_T - \phi_{i,j}}{\theta_T \Delta y} - \epsilon_{i,j-\theta_B/2}\frac{\phi_{i,j} - \phi_B}{\theta_B \Delta y}\right] \Big/ \left(\frac{\theta_T + \theta_B}{2}\Delta y\right) = -\rho_{i,j}, \end{aligned} \tag{27}$$

where $\phi_R, \phi_L, \phi_T, \phi_B$ are either cell center values, or interface values (when interface crossing).

3.8.5. Solving Poisson's Equation

In 2D, the normal derivative jump condition $[\epsilon\phi_n]$ are expressed in terms of Cartesian derivatives as $[\epsilon\phi_x]$ and $[\epsilon\phi_y]$. We have two different approaches to compute $[\epsilon\phi_x]$ and $[\epsilon\phi_y]$: One from the normal/tangential jumps and the other from a second-order polynomial reconstruction of ϕ in Ω^-. By equating these two different discretizations, we can express the interface values $\phi_R^-, \phi_L^-, \phi_T^-, \phi_B^-$ in terms of the neighboring cell values $\{\phi_{\mu\nu}\}$, where $\mu \in [i-2, i+2], \nu \in [j-2, j+2]$.

Let $\mathbf{x}_{\text{ext}} = (x_{\text{ext}}, y_{\text{ext}})$ be any of the four points $(x_{i-1}, y_{j-1}), (x_{i+1}, y_{j-1}), (x_{i-1}, y_{j+1}), (x_{i+1}, y_{j+1})$ belonging to Ω^-. ϕ_{ext} refers to the value of ϕ at $\mathbf{x}_{\text{ext}}$. Then, consider a quadratic polynomial $P^-(x, y) = Ax^2 + Bxy + Cy^2 + Dx + Ey + F$ that satisfies that interpolates:

$$\begin{aligned} P^-(\mathbf{x}_T) &= \phi_T^-, \\ P^-(\mathbf{x}_B) &= \phi_B^-, \\ P^-(\mathbf{x}_L) &= \phi_L^-, \\ P^-(\mathbf{x}_R) &= \phi_R^-, \\ P^-(x_i, y_j) &= \phi_{i,j}, \\ P^-(\mathbf{x}_{\text{ext}}) &= \phi_{\text{ext}}. \end{aligned} \tag{28}$$

We can always express the coefficients of polynomial P^- in terms of $\phi_T^-, \phi_B^-, \phi_L^-, \phi_R^-, \phi_{i,j}, \phi_{\text{ext}}$ by solving the above linear system. When there is no interface crossing, the interface values are just cell center values.

Hence, by using the equality $[pq] = [p]q^- + p^+[q]$, the Cartesian derivatives can be approximated by

$$\begin{aligned} [\epsilon\phi_x] &= [\epsilon\phi_n]n_x - [\epsilon\phi_\tau]n_y \\ &= [\epsilon\phi_n]n_x - [\epsilon]\phi_\tau^- n_y - \epsilon^+[\phi]_\tau n_y \\ &\approx [\epsilon\phi_n]n_x - [\epsilon]\left(\boldsymbol{\nabla}P^- \cdot \tau\right)n_y - \epsilon^+ a_\tau n_y. \end{aligned} \tag{29}$$

and similarly for $[\epsilon\phi_y]$,

$$\begin{aligned} [\epsilon\phi_y] &= [\epsilon\phi_n]n_y + [\epsilon\phi_\tau]n_x \\ &= [\epsilon\phi_n]n_y + [\epsilon]\phi_\tau^- n_x + \epsilon^+[\phi]_\tau n_x \\ &\approx [\epsilon\phi_n]n_y + [\epsilon]\left(\boldsymbol{\nabla}P^- \cdot \tau\right)n_x + \epsilon^+ a_\tau n_x. \end{aligned} \tag{30}$$

In the computation, b, n_x, n_y, a_τ are approximated by cubic interpolation of quantities defined at mesh cells.

On the other hand, the jumps $[\epsilon\phi_x]$ and $[\epsilon\phi_y]$ by definition are

$$\begin{aligned}[\epsilon\phi_x] &= \epsilon^+\phi_x^+ - \epsilon^-\phi_x^-,\\ [\epsilon\phi_y] &= \epsilon^+\phi_y^+ - \epsilon^-\phi_y^-.\end{aligned} \tag{31}$$

Therefore, $[\epsilon\phi_x]$ can be approximated by approximating ϕ^+ and ϕ^- using second order finite differences.

By equating the two different approximations of $[\epsilon\phi_x]$ and $[\epsilon\phi_y]$,

$$\begin{aligned}\epsilon^+\phi_x^+ - \epsilon^-\phi_x^- &\approx [\epsilon\phi_n]n_x - [\epsilon]\left(\boldsymbol{\nabla} P^- \cdot \tau\right) n_y - \epsilon^+ a_\tau n_y,\\ \epsilon^+\phi_y^+ - \epsilon^-\phi_y^- &\approx [\epsilon\phi_n]n_y - [\epsilon]\left(\boldsymbol{\nabla} P^- \cdot \tau\right) n_x - \epsilon^+ a_\tau n_x.\end{aligned} \tag{32}$$

we can express the interface values ϕ^- in terms of neighboring cell values. Similarly, by switching the role of Ω^- and Ω^+, we can also express the interface values ϕ^+ in terms of neighboring cell values. The final expressions are quite lengthy and thus omitted here for brevity.

Once we obtain the expression of $\phi_R^-, \phi_L^-, \phi_T^-, \phi_B^-$ in terms of neighboring cell values, we can substitute them into the modified discrete operator in Eq. (27) to obtain a linear system for $\boldsymbol{\Phi} = [..., \phi_{ij}, ...]^T$. This linear system is solved by a preconditioned GEMRES solver.

3.8.6. Computing Electric Field

After solving for the potential $\boldsymbol{\Phi}$, the electric field is computed by second-order central differences at cell centers when no adjacent node is across the interface,

$$\begin{aligned}E_x &= -\frac{\phi_{i+1,j} - \phi_{i-1,j}}{2\Delta x},\\ E_y &= -\frac{\phi_{i,j+1} - \phi_{i,j-1}}{2\Delta y}.\end{aligned} \tag{33}$$

When the adjacent node is across the interface, the electric field is computed by one-sided differences using the interface values, [2]

$$\begin{aligned}E_x &= -\frac{-\theta_R^2\phi_L + (\theta_R^2 - \theta_L^2)\phi_{i,j} + \theta_L^2\phi_R}{\theta_L\theta_R(\theta_L + \theta_R)\Delta x},\\ E_y &= -\frac{-\theta_T^2\phi_B + (\theta_T^2 - \theta_B^2)\phi_{i,j} + \theta_B^2\phi_T}{\theta_B\theta_T(\theta_B + \theta_T)\Delta y}.\end{aligned} \tag{34}$$

4. Numerical analysis of the scheme

4.1. Accuracy of the Vlasov IBM

Both mathematically show the accuracy of the constructed scheme

4.2. Accuracy of the Poisson GFM

Mathematically show the accuracy of the constructed scheme

5. Numerical verification and validation

5.1. Manufactured solution for the immersed Vlasov equation

Use linear advection convergence study to prove the accuracy proofed in 4.1

5.2. Manufactured solution for the Poisson equation

Use Poisson convergence study to prove the accuracy proofed in 4.2

5.3. Test benchmark

Sheath problem to show that the code actually work (or other simple test)

6. Application Study

A more realistic application case? (Compare with IFE-PIC)

7. Conclusion

References

[1] T. Xiong, J.-M. Qiu, Z. Xu, A. Christlieb, High order maximum principle preserving semi-lagrangian finite difference weno schemes for the vlasov equation, Journal of Computational Physics 273 (2014) 618–639. doi:https://doi.org/10.1016/j.jcp.2014.05.033.
URL https://www.sciencedirect.com/science/article/pii/S0021999114003969

[2] H. Cho, H. Han, B. Lee, Y. Ha, M. Kang, A second-order boundary condition capturing method for solving the elliptic interface problems on irregular domains, Journal of Scientific Computing 81 (1) (2019) 217–251. doi:10.1007/s10915-019-01016-y.